

جامعة نيويورك أبوظبي



NYU | ABU DHABI

Introduction to Artificial Intelligence

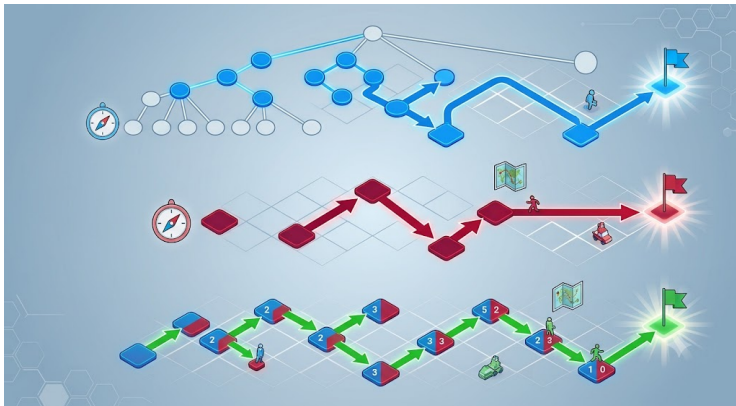
Informed Search

Julian Barreiro-Gomez

2026

Informed Search Strategies

- ▶ **Problem-specific knowledge** beyond the definition of the problem itself
- ▶ May find solutions **more efficiently**



Best-First Search

Best-First Search

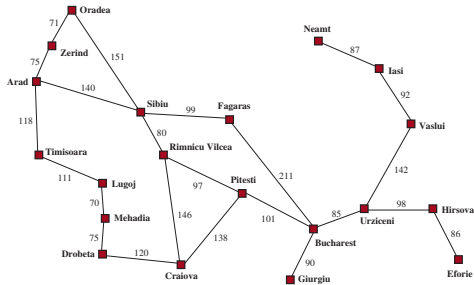
Some characteristics:

- ▶ **Tree-search** or **Graph-search** algorithm
- ▶ Node is selected for **expansion** based on an **evaluation function**, $f(n)$.
- ▶ The node with the **lowest evaluation is expanded first**
- ▶ The implementation of **best-first graph search** is **identical** to that for **Uniform-Cost Search (UCS)**
- ▶ Except for the **use of f instead of g** to order **priority queue**
- ▶ The **choice of f determines the search strategy**

Best-First Search

- Include as a component of f a **heuristic function**, denoted $h(n)$

$h(n)$ = estimated cost of the cheapest path from the state at node n to a goal state



Estimate cheapest path cost from Arad to Bucharest via **straight-line distance** Arad-Bucharest

- if n is goal node, then $h(n) = 0$

Greedy Best-First Search

Greedy Best-First Search

- Evaluates nodes using **heuristic function** i.e., $f(n) = h(n)$.
- Use the **straight line distance heuristic**, h_{SLD} .
- If **goal is Bucharest**, we need to know the **straight-line distances to Bucharest**

City	h_{SLD}	City	h_{SLD}
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

- $h_{SLD}(In(Arad)) = 366$

Greedy Best-First Search

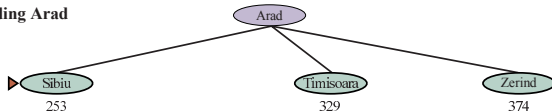
City	h_{SLD}	City	h_{SLD}	(a) The initial state
Arad	366	Mehadia	241	
Bucharest	0	Neamt	234	
Craiova	160	Oradea	380	
Drobeta	242	Pitesti	100	
Eforie	161	Rimnicu Vilcea	193	
Fagaras	176	Sibiu	253	
Giurgiu	77	Timisoara	329	
Hirsova	151	Urziceni	80	
Iasi	226	Vaslui	199	
Lugoj	244	Zerind	374	



Greedy Best-First Search

City	h_{SLD}	City	h_{SLD}
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

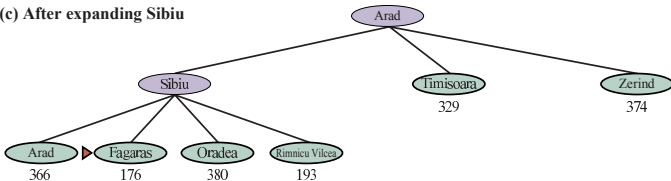
(b) After expanding Arad



Greedy Best-First Search

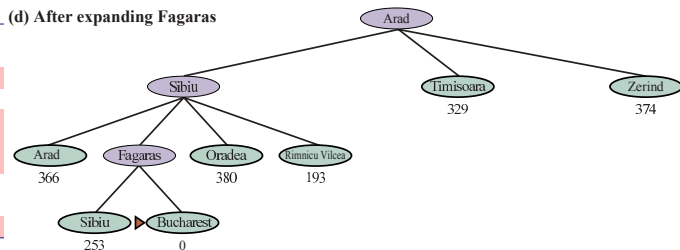
City	h_{SLD}	City	h_{SLD}
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

(c) After expanding Sibiu



Greedy Best-First Search

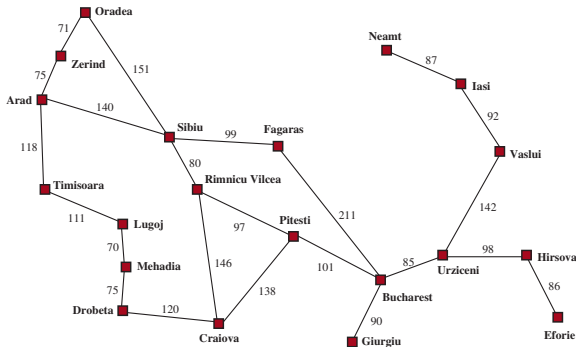
City	h_{SLD}	City	h_{SLD}
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



- ▶ Its search cost is minimal
- ▶ Greedy best-first tree search is also **incomplete** even in a finite state space, much like **depth-first search**

Greedy Best-First Search

Consider the problem **from Iasi to Fagaras**. The heuristic suggests that **Neamt** must be expanded first because it is **closest to Fagaras**, but it is a **dead end**.



- ▶ The solution is to go **first to Vaslui** (actually farther from the goal)
- ▶ The algorithm **will never find** this solution

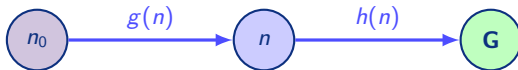
A* Search

A* Search

It evaluates nodes by combining

1. **cost to reach** the node $g(n)$
2. **cost to get from** the node to the goal $h(n)$

Thus, $f(n) = g(n) + h(n)$.

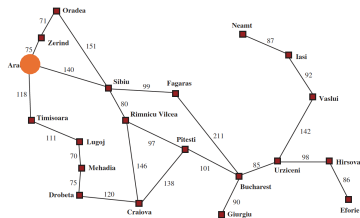


$f(n)$ = estimated cost of the cheapest solution through n .

Given **heuristic function** $h(n)$ **satisfies** certain conditions, A* search is **complete and optimal**

Algorithm is **identical to UNIFORM-COST-SEARCH** except that **A*** uses $g + h$ instead of g .

Stages in an A* Search for Bucharest

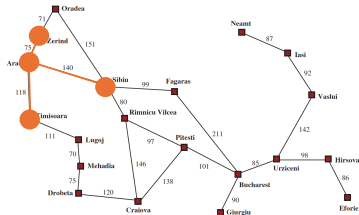


(a) The initial state

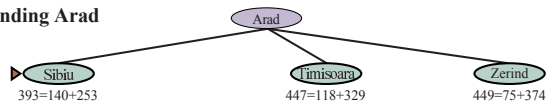
Arad
366=0+366

City	h_{SLD}	City	h_{SLD}
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Stages in an A* Search for Bucharest

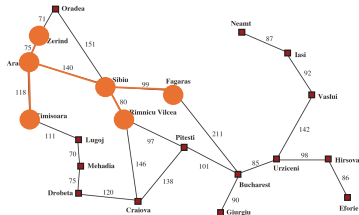


(b) After expanding Arad

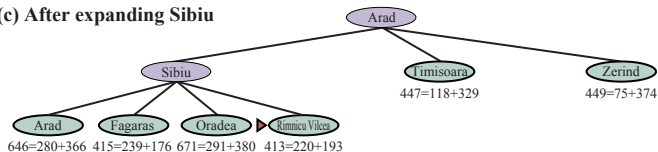


City	h_{SLD}	City	h_{SLD}
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Stages in an A* Search for Bucharest

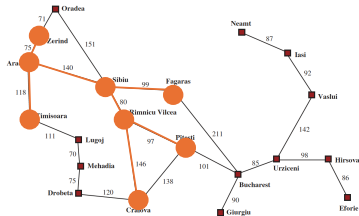


(c) After expanding Sibiu

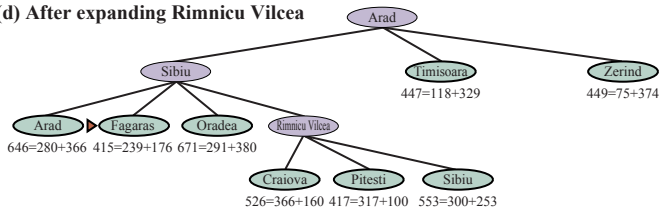


City	h_{SLD}	City	h_{SLD}
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Stages in an A* Search for Bucharest

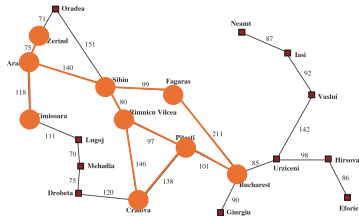


(d) After expanding Rimnicu Vilcea

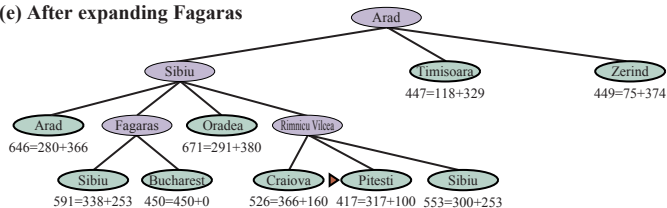


City	h_{SLD}	City	h_{SLD}
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Stages in an A* Search for Bucharest

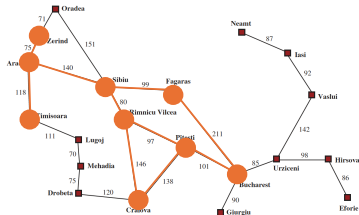


(e) After expanding Fagaras

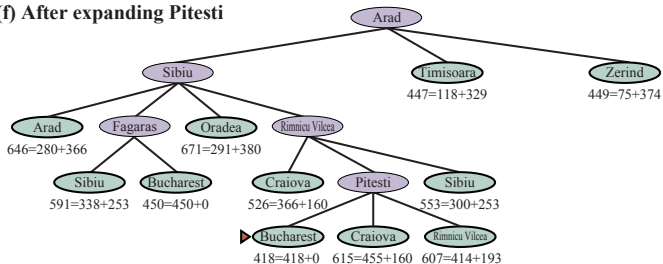


City	h_{SLD}	City	h_{SLD}
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Stages in an A* Search for Bucharest



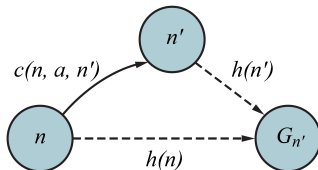
(f) After expanding Pitesti



For **Optimality**, $h(n)$ must be an **admissible heuristic**.
Never overestimates the cost to the goal.

City	h_{SLD}	City	h_{SLD}
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

A* Search



Triangle inequality

If the heuristic h is **consistent**, then the single number $h(n)$ will be less than the sum of the cost $c(n, a, n')$ of the action from n to n' plus the heuristic estimate $h(n')$.

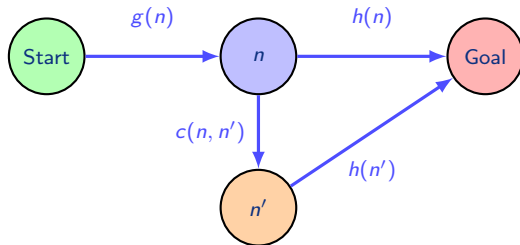
For **optimality**, slightly stronger condition called **consistency** (or sometimes **monotonicity**) is required.

$$h(n) \leq c(n, a, n') + h(n')$$

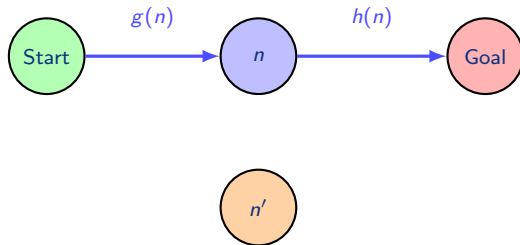
Optimality of A*

If $h(n)$ is consistent, then the values of $f(n)$ along any path are nondecreasing

Consistency: $h(n) \leq c(n, n') + h(n')$ where n' is a successor of n .

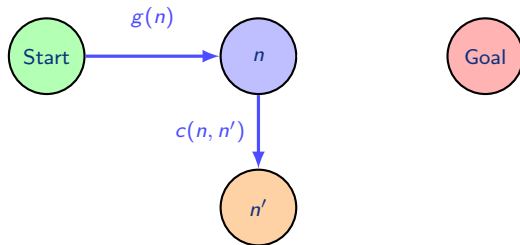


Optimality of A*



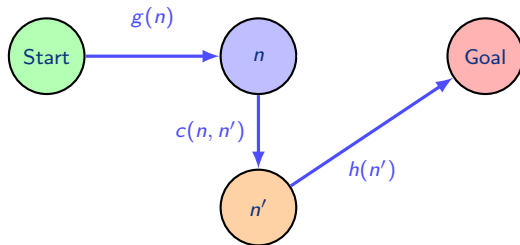
$$f(n) = g(n) + h(n),$$

Optimality of A*



$$f(n) = g(n) + h(n),$$
$$g(n') = g(n) + c(n, n'),$$

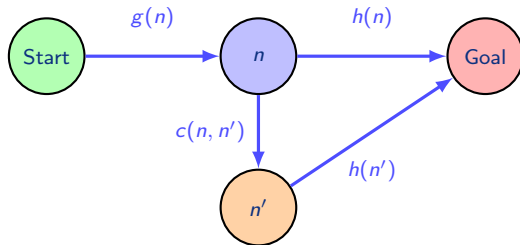
Optimality of A*



$$\begin{aligned}f(n) &= g(n) + h(n), \\g(n') &= g(n) + c(n, n'),\end{aligned}$$

$$\begin{aligned}f(n') &= g(n') + h(n'), \\f(n') &= g(n) + c(n, n') + h(n'),\end{aligned}$$

Optimality of A*



$$f(n) = g(n) + h(n),$$

$$f(n') = g(n') + h(n'),$$

$$g(n') = g(n) + c(n, n'),$$

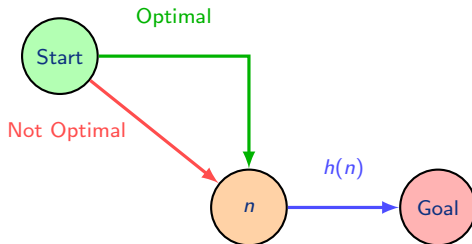
$$f(n') = g(n) + c(n, n') + h(n'),$$

and using the consistency $h(n) \leq c(n, n') + h(n')$ yields

$$f(n') \geq g(n) + h(n) = f(n), \implies f(n') \geq f(n)$$

Optimality of A*

Claim. Whenever **A*** selects a node n for expansion, the optimal path to that node has been found. (Optimality)

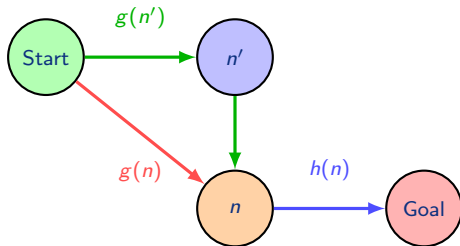


Optimal Path

Not Optimal Path

Optimality of A*

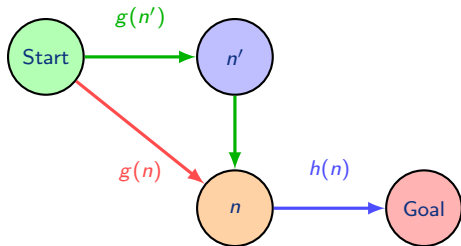
Claim. Whenever **A*** selects a node n for expansion, the optimal path to that node has been found. (Optimality)



Optimal Path

Not Optimal Path

Optimality of A*



Note that

$$f^*(n') \leq f^*(n) \text{ (nondecreasing } f(n))$$

Also

$$f^*(n) < f(n) \text{ (optimality of } f^*(\cdot))$$

It follows $f^*(n') < f(n)$.

- ▶ Suppose that the **“not optimal path”** is selected to node n .
- ▶ There must be a node n' in the **optimal path** which is in the **frontier list**
- ▶ Let $f^*(n)$ be the cost from Start to n (**optimal path**)
- ▶ Let $f(n)$ be the cost from Start to n (**not optimal path**)
- ▶ Let $f^*(n')$ be the cost from Start to n' (**optimal path**)

In the front list $f^*(n')$ should've been selected instead of $f(n)$

Thank you